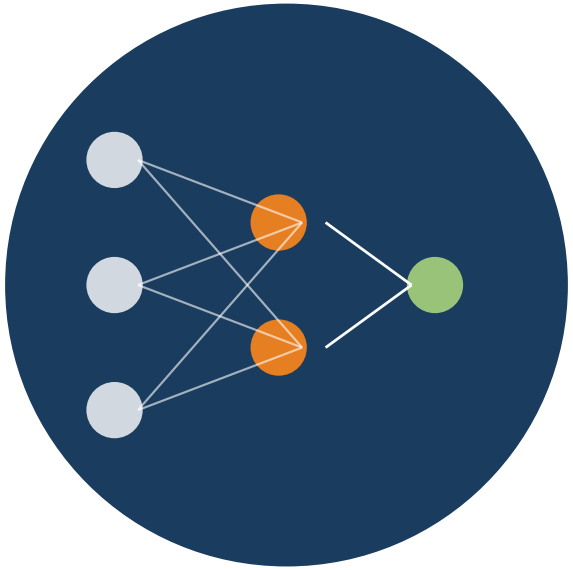


Redes Neurais Multicamadas

com PyTorch `torch.nn`

Deep Learning: Regressão, Classificação Binária e Multiclasse

Aula 6

Prof. Dr. Rooney Coelho

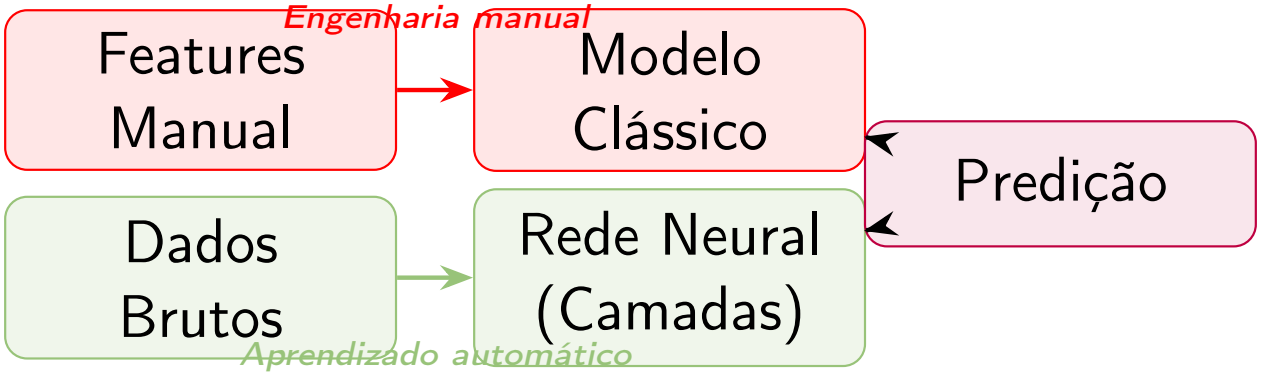
22 de março de 2026

Objetivos da Aula

1. **Paradigma Deep Learning:** Representações hierárquicas vs engenharia manual
2. **Módulo `torch.nn`:** De tensores básicos a módulos de alto nível
3. **Implementação prática:** MLPs para Regressão, Classificação Binária e Multiclasse
4. **Seleção adequada:** Funções de perda e ativações para cada tarefa

*Referência principal: "**Deep Learning with PyTorch**" (Stevens et al.)*

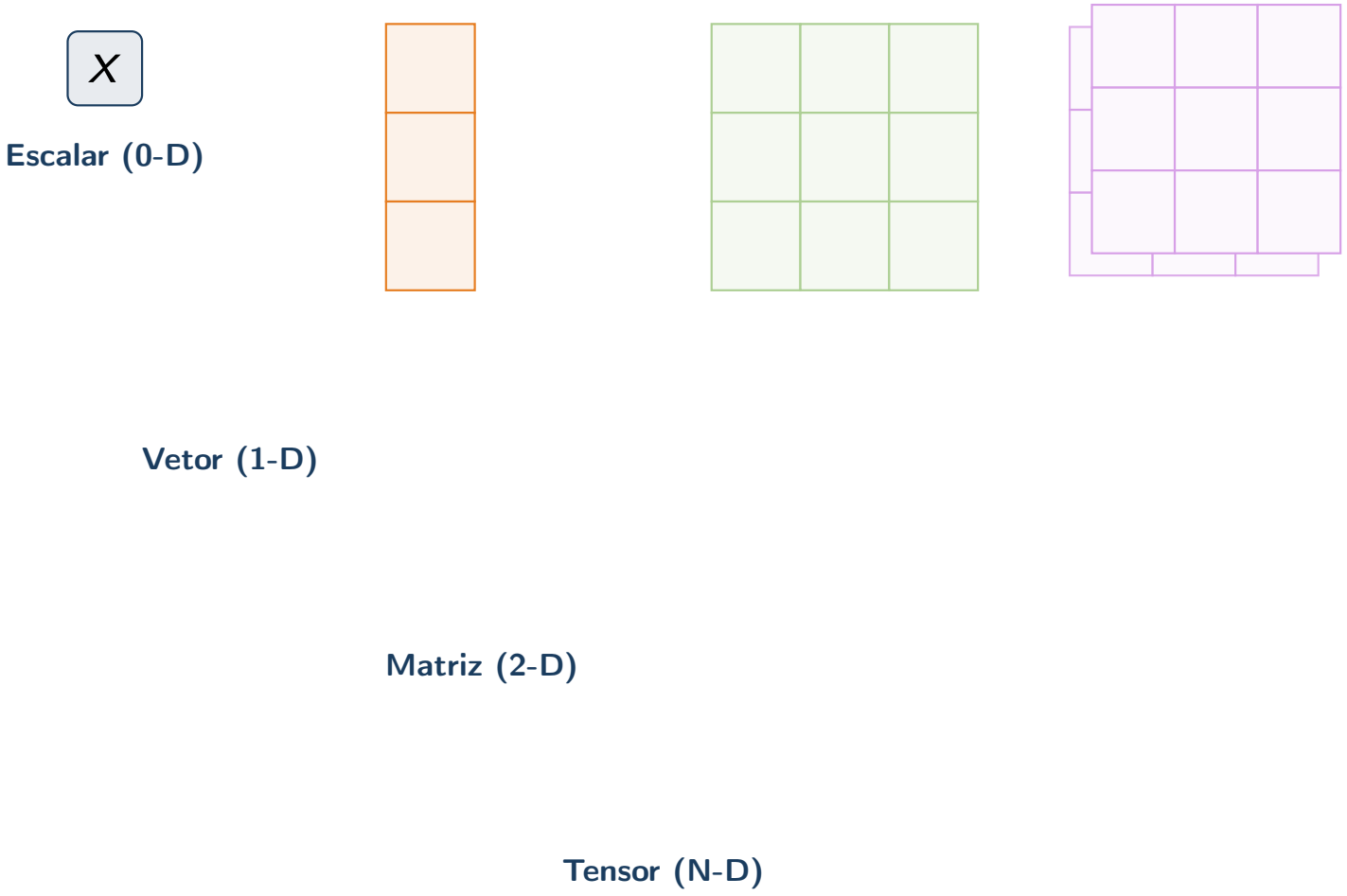
O Paradigma do Deep Learning



Conceito Chave

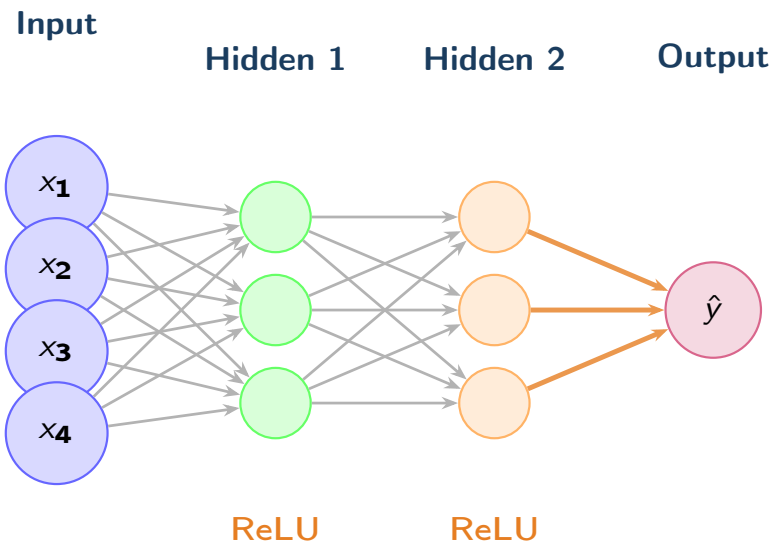
Deep Learning aprende **representações hierárquicas** diretamente dos dados, dispensando a engenharia manual de características.

Hierarquia de Tensores



Tensores generalizam escalares, vetores e matrizes para dimensões arbitrárias.

Arquitetura de uma Rede Neural Multicamadas (MLP)



$\mathrm{h} = \sigma(\mathrm{Wx} + \mathrm{b})$ — x entrada, W pesos, b viés, σ ativação

Implementação: nn.Sequential

```python
import torch
import torch.nn as nn

# Definir modelo MLP com nn.Sequential
model = nn.Sequential(
    nn.Linear(input_dim, hidden_dim),  # Camada oculta 1
    nn.ReLU(),                         # Ativacao nao-linear
    nn.Linear(hidden_dim, hidden_dim), # Camada oculta 2
    nn.ReLU(),
    nn.Linear(hidden_dim, output_dim)  # Camada saida
)

# Forward pass
x = torch.randn(batch_size, input_dim)
y_pred = model(x)
```

O **nn.Sequential** agrupa camadas em ordem sequencial — ideal para MLPs simples.

Preparação: Divisão Treino/Teste

- **Treino (~80%)**: atualizar pesos
- **Teste (~20%)**: avaliar desempenho
- Previne **overfitting** — memorização vs generalização

Preparação: Normalização com StandardScaler

- Cada feature: média ≈ 0, desvio padrão ≈ 1
- Otimização **muito mais rápida**

$$x'_{\text{norm}} = \frac{x - \mu}{\sigma}$$

Calcular estatísticas **apenas no treino**!

Aplicação 1: Regressão — Preços

Objetivo: Prever um valor contínuo (ex: preço de casa)

Configuração:

- **Output**: `nn.Linear(hidden_dim, 1)` — sem ativação
- **Loss**: `nn.MSELoss()`
- **Otimizador**: Adam

Regressão: MSE Loss e Métricas

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2$$

Métricas:

- MSE, RMSE
- MAE (erro médio absoluto)
- R^2 (coeficiente de determinação)

Regressão: Preparação de Dados

```python
import torch.nn as nn
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Gerar dados sint ticos
np.random.seed(42)
X = np.random.randn(300, 5)
y = 3*X[:,0] + 2*X[:,1] - X[:,2] + np.random.randn(300)*0.1

# Divis o Treino/Teste
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Normaliza  o
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

X_train = torch.FloatTensor(X_train)
y_train = torch.FloatTensor(y_train).reshape(-1, 1)
X_test  = torch.FloatTensor(X_test)
y_test  = torch.FloatTensor(y_test).reshape(-1, 1)
```

Regressão: Loop de Treino

```python
# Modelo
model = nn.Sequential(
    nn.Linear(5, 64),
    nn.ReLU(),
    nn.Linear(64, 32),
    nn.ReLU(),
    nn.Linear(32, 1)
)

# Loss e Otimizador
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# Treino
epochs = 100
for epoch in range(epochs):
    # Forward
    y_pred = model(X_train)
    loss = criterion(y_pred, y_train)

    # Backward
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Aplicação 2: Classificação Binária

Objetivo: Prever classe entre 2 opções (ex: sim/não)

Configuração:

- **Output:** `nn.Linear(hidden_dim, 1)`
- **Ativação:** `Sigmoid`
- **Loss:** `nn.BCELoss()`
- **Decisão:** $\hat{y} \geq 0.5 \Rightarrow$ classe 1

Classificação Binária: BCE Loss e Métricas

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\right]$$

Métricas: Acurácia, Precisão, Recall, F1-Score, AUC

Aplicação 3: Classificação Multiclasse

Objetivo: Prever classe entre K opções (ex: 3 espécies)

Configuração:

- **Output:** `nn.Linear(hidden_dim, num_classes)`
- **Loss:** `nn.CrossEntropyLoss()`
- **Predição:** argmax dos logits

Multiclasse: Softmax e CrossEntropy Loss

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad \mathcal{L}_{\text{CE}} = -\sum_{i=1}^{N} y_i \log(\hat{y}_i)$$

Métricas: Acurácia, Matriz de Confusão

Comparação: Loss por Tarefa

Tarefa	**Output**	**Loss**
Regressão	1 valor real	MSELoss
Binária	1 valor [0,1]	BCELoss
Multiclasse	K logits	CrossEntropyLoss

Funções de Ativação (1/2)

ReLU (Rectified Linear Unit):

$$\text{ReLU}(x) = \max(0, x)$$

- Rápido, simples, muito eficaz
- Padrão em Deep Learning moderno
- "Dying ReLU"quando muitos neurônios ficam zero

Tanh:

$$\text{Tanh}(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$$

- Output $\in [-1, 1]$, zero-centrado
- Mais lento que ReLU

Funções de Ativação (2/2)

Sigmoid:

$$\text{Sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

- Output $\in [0, 1]$ — ideal para probabilidades
- Use para **classificação binária**
- Gradientes desaparecem em extremos

Fluxo Geral de Treinamento

```python
# 1. Prepara  o
model = MeuModelo()
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters())

# 2. Loop de Treinamento
for epoch in range(num_epochs):
    for X_batch, y_batch in train_loader:
        # Forward pass
        y_pred = model(X_batch)
        loss = criterion(y_pred, y_batch)

        # Backward pass
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    # Avalia  o no teste
    with torch.no_grad():
        y_test_pred = model(X_test)
        test_loss = criterion(y_test_pred, y_test)
        print(f"Epoch {epoch}: train_loss={loss:.4f}, test_loss={test_loss:.4f}")
```

Dicas Práticas para Treinar MLPs (1/2)

1. **Normalizar os dados** — melhora convergência drasticamente
2. **Usar Adam** — otimizador muito mais robusto que SGD puro
3. **Monitorar Loss** — plot em log pode revelar problemas

Dicas Práticas para Treinar MLPs (2/2)

1. **Batch Normalization** — estabiliza o treino
   - `nn.BatchNorm1d(num_features)` para dados tabulares
2. **Dropout** — reduz overfitting
   - `nn.Dropout(p=0.5)` — desativa 50% dos neurônios
3. **Early Stopping** — parar quando loss no teste para de melhorar

Conclusão

Resumo

Você aprendeu a **construir, treinar e avaliar** redes neurais com PyTorch para **Regressão**, **Classificação Binária** e **Multiclasse**.

Próximos Passos:

- **Arquiteturas mais complexas:** CNNs (Convolução), RNNs (Sequências)
- **Transfer Learning:** Usar modelos pré-treinados
- **Regularização avançada:** Batch Norm, Dropout, Data Augmentation
- **Debugging:** TensorBoard, visualização de ativações

Obrigado!

Referências

1. Stevens, E., Viering, T., & Bethge, M. (2024). *Deep Learning with PyTorch*. Manning Publications.
2. PyTorch Official Documentation: `https://pytorch.org`
3. TensorFlow Hub (Transfer Learning): `https://www.tensorflow.org/hub`
4. Goodfellow, I., Bengio, Y., & Courville, A. *Deep Learning*. MIT Press.

Código disponível em:
GitHub / Colab